

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA0706

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:25

Annexure A

Coding challenge

Code of Conduct:

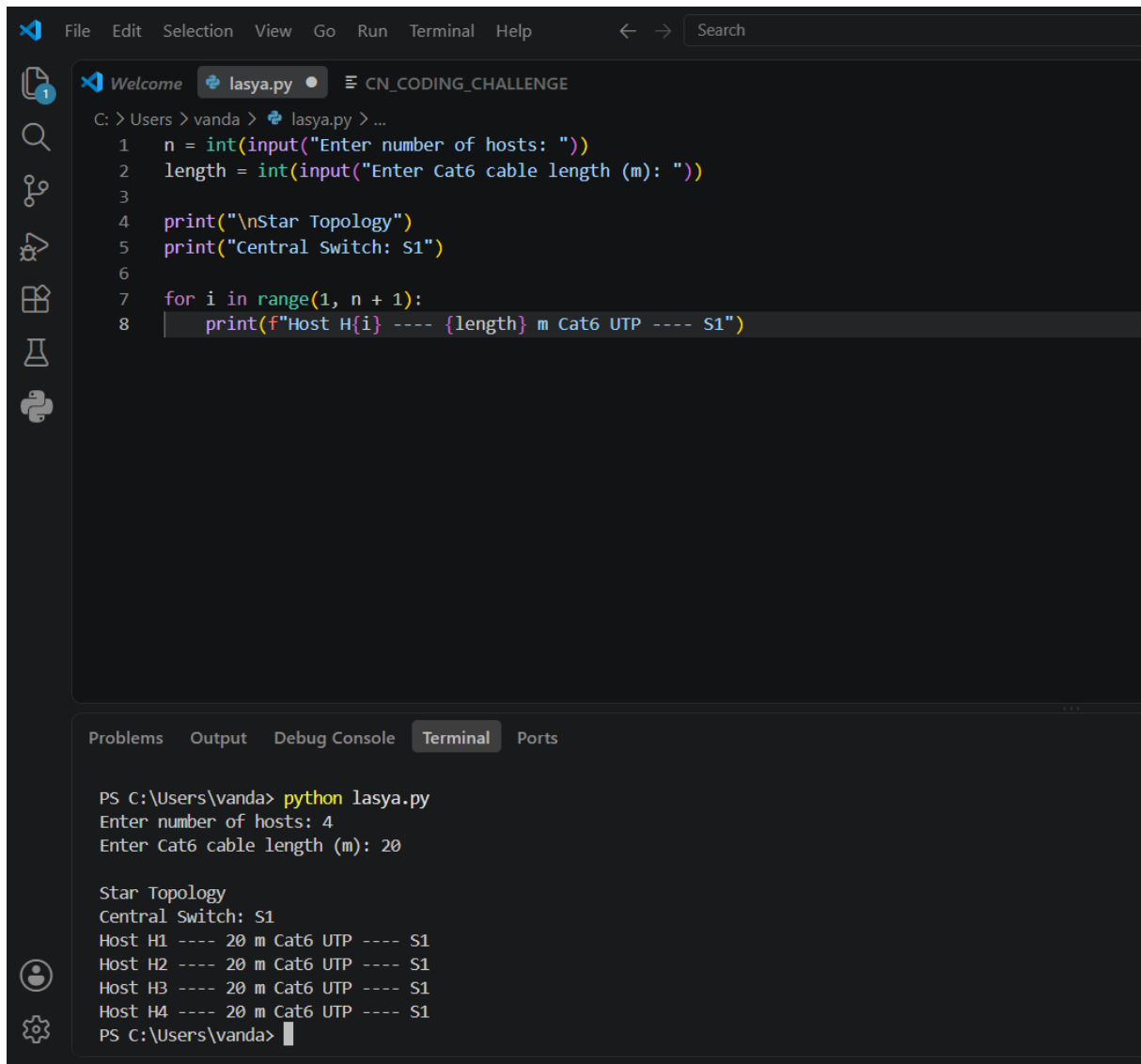
I _____V.Lasya(192524186)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: v.lasya

Annexure A

Coding challenge

1. Write a Python script that generates a star topology diagram (ASCII or graphical) given n hosts and a central switch. The script must print each host label, the switch name, and the connecting Cat6 UTP cable length.



The image shows a Visual Studio Code editor window with a Python script named `lasya.py` and its execution output in the terminal.

Python Script (`lasya.py`):

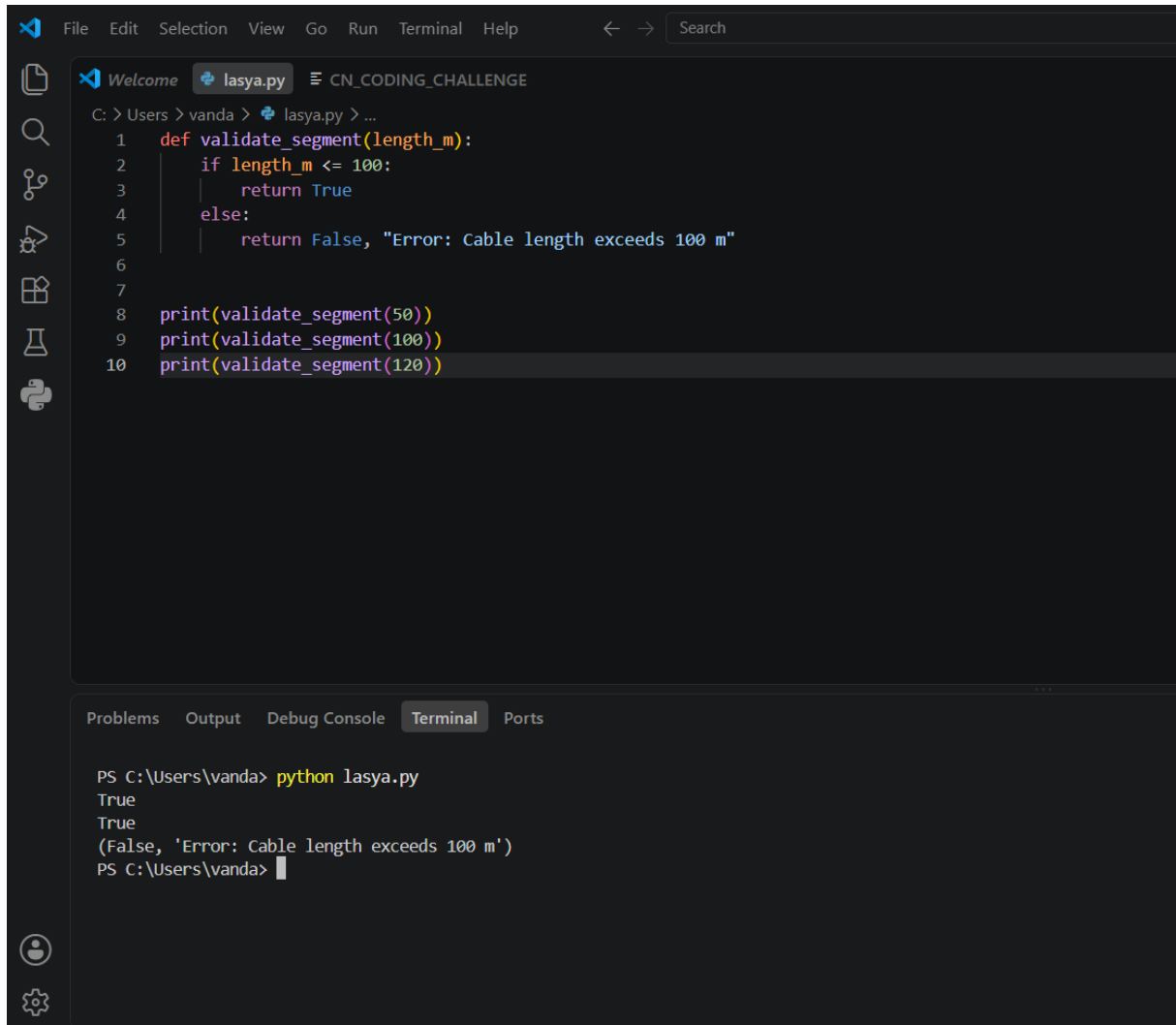
```
C: > Users > vanda > lasya.py > ...
1  n = int(input("Enter number of hosts: "))
2  length = int(input("Enter Cat6 cable length (m): "))
3
4  print("\nStar Topology")
5  print("Central Switch: S1")
6
7  for i in range(1, n + 1):
8      print(f"Host H{i} ---- {length} m Cat6 UTP ---- S1")
```

Terminal Output:

```
PS C:\Users\vanda> python lasya.py
Enter number of hosts: 4
Enter Cat6 cable length (m): 20

Star Topology
Central Switch: S1
Host H1 ---- 20 m Cat6 UTP ---- S1
Host H2 ---- 20 m Cat6 UTP ---- S1
Host H3 ---- 20 m Cat6 UTP ---- S1
Host H4 ---- 20 m Cat6 UTP ---- S1
PS C:\Users\vanda>
```

2. Code a function `validate_segment(length_m)` that returns True if the UTP segment is within IEEE 802.3 limits (≤ 100 m) and False with an error message otherwise. Include at least 3 test cases.



The image shows a Visual Studio Code editor window with a Python file named `lasya.py`. The code defines a function `validate_segment` that checks if a cable length is within the IEEE 802.3 limit of 100 meters. It includes three test cases: 50m, 100m, and 120m. Below the editor, a terminal window shows the execution of the script, which outputs `True` for 50m and 100m, and `(False, 'Error: Cable length exceeds 100 m')` for 120m.

```
1 def validate_segment(length_m):
2     if length_m <= 100:
3         return True
4     else:
5         return False, "Error: Cable length exceeds 100 m"
6
7
8 print(validate_segment(50))
9 print(validate_segment(100))
10 print(validate_segment(120))
```

```
PS C:\Users\vanda> python lasya.py
True
True
(False, 'Error: Cable length exceeds 100 m')
PS C:\Users\vanda>
```

3. Write a script that simulates CSMA/CD on a star topology with 4 hosts. Log every collision detection event and the back-off interval to a file named 'csma_log.txt'.

The image shows a VS Code editor with a Python script named 'lasya.py' in the editor pane. The script simulates CSMA/CD on a star topology with 4 hosts. It uses a random backoff interval (1 to 5 seconds) for collision detection and logs the results to 'csma_log.txt'. The terminal pane shows the output of the script, including collision detection events and successful transmissions for hosts H1, H2, H3, and H4. A separate window titled 'csma' displays the log file content, showing the sequence of events and back-off intervals for each host.

```
12
13
14     if collision:
15         backoff = random.randint(1, 5)
16
17         message = f"(host): Collision detected, Back-off = {backoff} seconds"
18
19         print(message)
20         file.write(message + "\n")
21
22     else:
23         message = f"(host): Data transmitted successfully"
24
25         print(message)
26         file.write(message + "\n")
27
28 print("\nSimulation completed.")
29 print("Log saved in csma_log.txt")
```

Terminal Output:

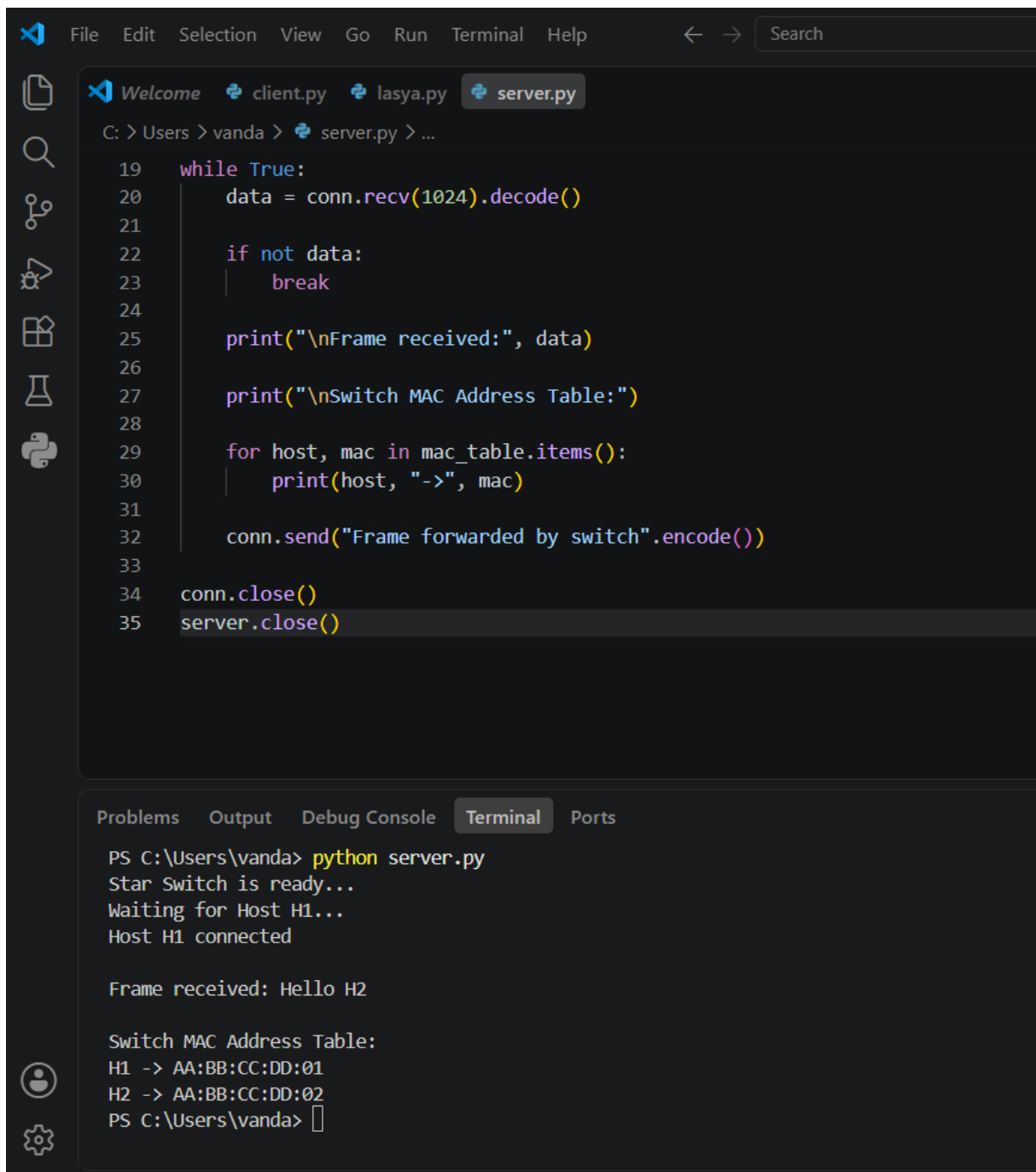
```
H3: Collision detected, Back-off = 3 seconds
H1: Collision detected, Back-off = 2 seconds
H2: Data transmitted successfully
H3: Collision detected, Back-off = 3 seconds
H1: Data transmitted successfully
H4: Data transmitted successfully

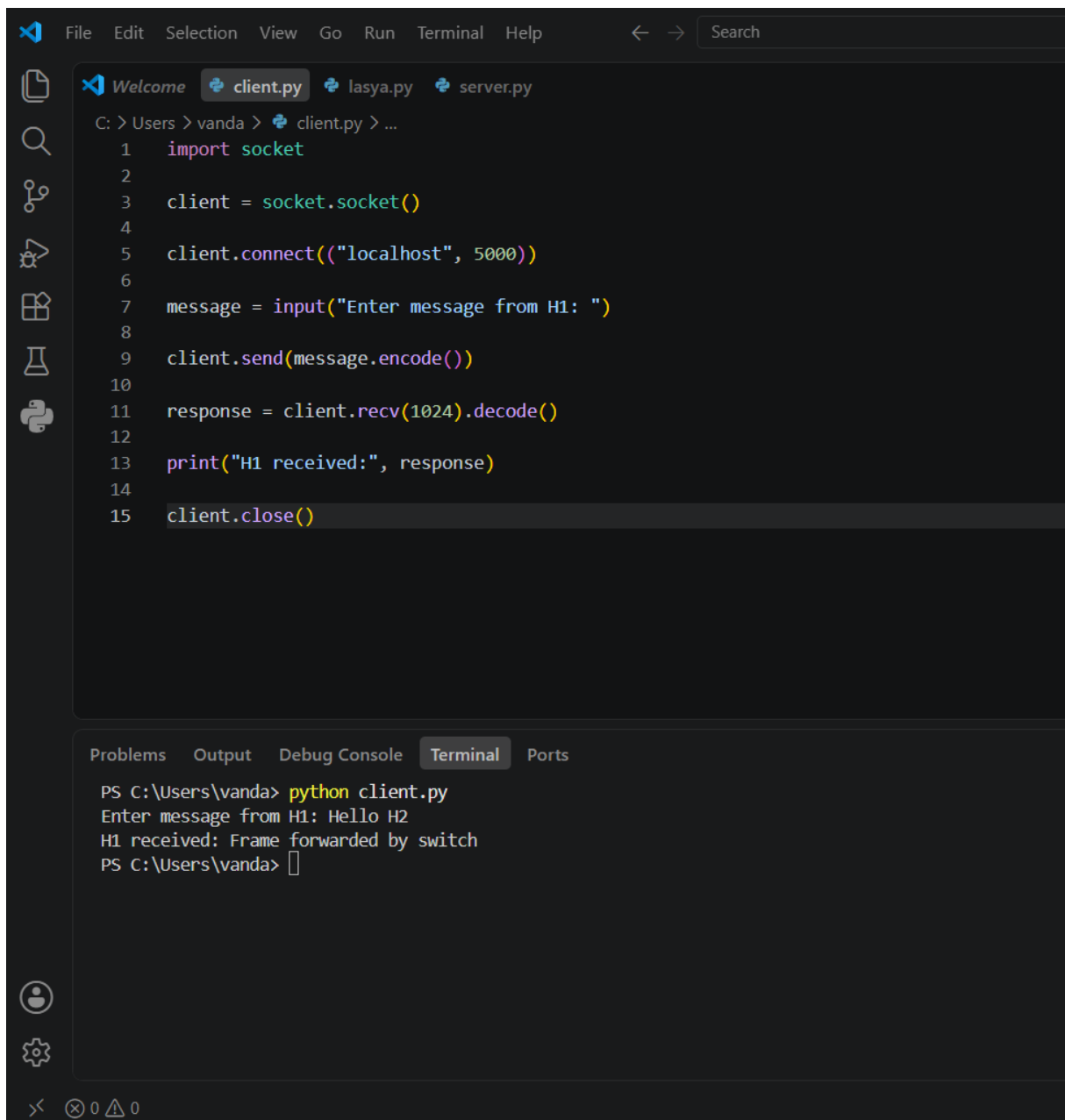
Simulation completed.
Log saved in csma_log.txt
PS C:\Users\vanda> dir csma_log.txt
```

csma Log File Content:

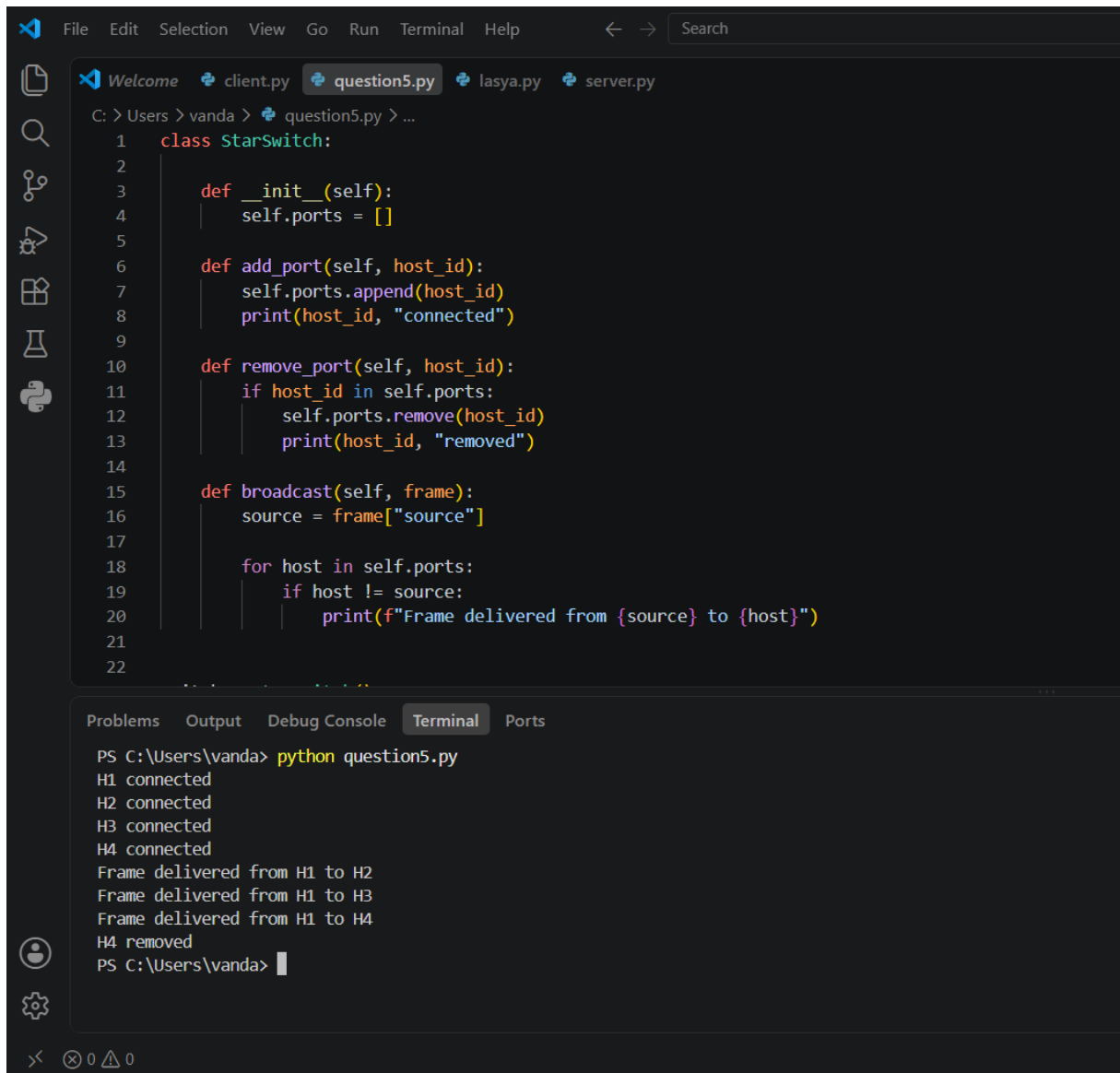
```
H4: Data transmitted successfully
H1: Collision detected, Back-off = 3 seconds
H1: Data transmitted successfully
H2: Collision detected, Back-off = 5 seconds
H3: Collision detected, Back-off = 3 seconds
H1: Collision detected, Back-off = 2 seconds
H2: Data transmitted successfully
H3: Collision detected, Back-off = 3 seconds
H1: Data transmitted successfully
H4: Data transmitted successfully
```

4. Using Python's 'socket' library, write a client-server program that models two hosts communicating through a star switch. Demonstrate message forwarding by printing the switch's MAC address table after each frame transmission.





5. Create a class `StarSwitch` with methods `add\_port(host\_id)`, `remove\_port(host\_id)`, and `broadcast(frame)`. Broadcast must send a frame to all ports except the source and log each delivery.



The image shows a Visual Studio Code editor window with a dark theme. The file explorer on the left shows a project with files: `Welcome`, `client.py`, `question5.py` (selected), `lasya.py`, and `server.py`. The editor displays the code for `question5.py`, which defines a `StarSwitch` class. The class has three methods: `__init__` (initializes `self.ports` as an empty list), `add_port` (appends `host_id` to `self.ports` and prints "connected"), and `remove_port` (removes `host_id` from `self.ports` if it exists and prints "removed"). The `broadcast` method takes a `frame` as input, extracts the `source` from the frame, and iterates over `self.ports` to deliver the frame to all hosts except the source, printing the delivery message for each.

```
1 class StarSwitch:
2
3     def __init__(self):
4         self.ports = []
5
6     def add_port(self, host_id):
7         self.ports.append(host_id)
8         print(host_id, "connected")
9
10    def remove_port(self, host_id):
11        if host_id in self.ports:
12            self.ports.remove(host_id)
13            print(host_id, "removed")
14
15    def broadcast(self, frame):
16        source = frame["source"]
17
18        for host in self.ports:
19            if host != source:
20                print(f"Frame delivered from {source} to {host}")
21
22
```

Below the editor, the `Terminal` tab is active, showing the output of running `python question5.py` in a PowerShell prompt. The output shows four hosts (H1, H2, H3, H4) connecting, followed by three broadcast messages from H1 to H2, H3, and H4, and finally H4 disconnecting.

```
PS C:\Users\vanda> python question5.py
H1 connected
H2 connected
H3 connected
H4 connected
Frame delivered from H1 to H2
Frame delivered from H1 to H3
Frame delivered from H1 to H4
H4 removed
PS C:\Users\vanda>
```

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done